

1 Unidade 1 - A Importância e os Benefícios da Modelagem.

A modelagem de sistemas é uma etapa fundamental no desenvolvimento de software de qualidade. Ela serve como uma ferramenta estratégica para mitigar riscos, otimizar processos e garantir a entrega de um produto adequado às demandas reais.

1.1. Evitar Retrabalho e Prejuízos:

O uso de modelos evita o estresse e o custo financeiro decorrentes do desenvolvimento de componentes errados ou inconsistentes com o que o cliente realmente precisa.

1.2. Comunicação Efetiva:

Atua como um canal comum de comunicação, facilitando o entendimento mútuo entre os desenvolvedores, os usuários e os demais stakeholders. Previsibilidade:

Ajuda a construir soluções com maior previsibilidade de tempo e custos nas fases posteriores do projeto.

1.3. Uso de Notações:

A utilização de padrões de notações (notação gráfica padronizada), embora exija um aprendizado inicial por parte dos leitores, enriquece a documentação ao captar decisões estratégicas e semânticas complexas que não são óbvias apenas com a leitura do código-fonte.

2. Princípios de Modelagem

De acordo com os autores clássicos Booch, Rumbaugh e Jacobson, a atividade de planejamento e modelagem de sistemas é guiada por quatro princípios básicos essenciais:

2.1. A escolha dos modelos influencia a abordagem do problema:

A forma como o projetista escolhe modelar o sistema define diretamente como a solução será estruturada. Projetistas diferentes podem criar soluções distintas a partir do mesmo conjunto de requisitos, dependendo da sua visão e experiência.

2.2. Níveis de precisão variados:

Um sistema complexo exige a capacidade de alternar entre uma visão panorâmica (macro) e o detalhamento da base de componentes (micro). Bons modelos permitem aplicar essa variação de "zoom" com facilidade.

2.3. Conexão com a realidade:

Os modelos devem ser intuitivos e fáceis de associar com o mundo físico ou as regras reais do negócio.

2.4. Ausência de um modelo único isolado:

Sistemas robustos não podem ser explicados de forma completa por apenas um ponto de vista. É necessário um conjunto de múltiplos modelos independentes, mas complementares, para cobrir todas as nuances da aplicação.

3. Atividades no Processo de Análise e Projeto

A engenharia e o desenvolvimento de software passam por etapas sistemáticas fundamentais apresentadas nesta unidade:

3.1. Levantamento de Requisitos:

Processo de investigação inicial para descobrir e definir detalhadamente as necessidades do cliente e o que o software precisa realizar.

3.2. Análise, Validação e Verificação de Modelos:

Fase em que se investiga e valida a consistência do que será feito, assegurando que os modelos estão corretos e correspondem ao negócio antes de se iniciar a codificação.

3.3. Projeto (Desenho):

Etapa de detalhamento técnico onde são desenhadas as soluções arquiteturais do sistema, especificando as características dos componentes de software.

3.4. Implementação, Testes e Implantação:

O momento de traduzir os designs em código real, submeter o produto a baterias de testes para garantir a ausência de falhas e, por fim, colocá-lo no ambiente de produção para uso.

3.5. Manutenção e Evolução:

Atualizações contínuas necessárias para corrigir problemas descobertos em uso ou para adaptar o software a novos cenários comerciais e tecnológicos.

4. Principais Participantes do Processo

O desenvolvimento de software é uma atividade multidisciplinar e conta com papéis fundamentais de acordo com a área de atuação:

4.1. Administrador de Banco de Dados (DBA):

Responsável por definir tabelas, otimizar o desempenho das bases de dados e garantir a integridade de todas as informações gravadas.

4.2. Especialista de Domínio:

A pessoa que detém o conhecimento aprofundado sobre a área de negócio em que o software está sendo aplicado (muitas vezes representado pelo próprio cliente-usuário). Ele se diferencia do cliente-contratante, que é quem financia e encomenda o projeto (e que pode não conhecer a fundo as rotinas operacionais).

4.3. Avaliadores de Qualidade:

Profissionais responsáveis por analisar se o produto e os processos de desenvolvimento seguem estritamente os padrões estabelecidos.

(Nota prática da apostila: Em pequenas ou microempresas, é muito comum que esses papéis se acumulem, com analistas programando, programadores fazendo análises e gerentes de projeto atuando na avaliação da qualidade).

EXEMPLO PRÁTICO

vamos aplicar cada um dos tópicos a um exemplo prático: o desenvolvimento de um **Sistema de Controle de Estoque e Vendas para uma Livraria**.

1. A Importância e os Benefícios da Modelagem

Imagine que o dono da livraria pediu um sistema para "controlar as vendas". Se você começar a programar imediatamente sem modelar, pode entender que o foco são apenas as vendas no balcão.

- **Evitando Retrabalho:** Ao criar um modelo visual simples do fluxo (um diagrama) e mostrar ao cliente, ele percebe e diz: *"Mas eu também preciso que o sistema avise quando um livro atingir menos de 5 unidades no estoque para eu falar com a editora"*. Se você não tivesse modelado isso antes, teria que refazer grande parte do banco de dados e do código depois do sistema pronto, gerando prejuízo e perda de tempo.
- **Comunicação Efetiva:** O desenho do fluxo de venda permite que o programador, o dono da livraria e o caixa entendam exatamente a mesma regra: o livro só baixa do estoque *depois* que o pagamento for aprovado.

2. Princípios de Modelagem

Ao planejar o sistema da livraria, aplicamos os quatro princípios:

1. **A escolha do modelo influencia a abordagem:** Se decidirmos modelar o sistema focando primeiro no fluxo financeiro (caixa), o software terá uma estrutura diferente do que se focarmos primeiro na experiência de busca de livros pelo cliente. A escolha do foco dita o caminho.
2. **Níveis de precisão variados (Zoom):** * *Visão Macro (Visão panorâmica):* Um diagrama simples mostrando que o cliente escolhe o livro, o caixa registra e o estoque atualiza.
 - *Visão Micro (Detalhada):* Um modelo que especifica exatamente o que acontece quando o cartão de crédito é recusado e quais dados do cliente (CPF, Nome, Cartão) são transmitidos.
3. **Conexão com a realidade:** No modelo do sistema, a entidade "Livro" terá propriedades reais como *Título, Autor, ISBN e Preço de Venda*, facilitando a associação com o produto físico que está na prateleira.
4. **Ausência de um modelo único:** Para a livraria funcionar, você precisará de um modelo para o banco de dados, outro modelo para o comportamento da tela do caixa (interface) e outro para o processo de entrega de fornecedores.

3. Atividades no Processo de Análise e Projeto

Na prática, o desenvolvimento do sistema da livraria seguiria estes passos:

- **Levantamento de Requisitos:** Você vai até a livraria, entrevista os funcionários e anota: *"O sistema deve aceitar cartão e dinheiro"*, *"O sistema deve emitir nota fiscal"*.
- **Análise e Validação:** Você analisa as anotações e os modelos criados para garantir que não há contradições (ex: uma regra dizendo que o cliente pode levar o livro sem pagar confrontando outra que diz que o pagamento é obrigatório).
- **Projeto (Desenho):** Você define a arquitetura técnica: o sistema rodará em um navegador web? Usará qual banco de dados? Como será o desenho das telas?
- **Implementação e Testes:** Você digita o código do sistema e faz testes simulados: tenta vender um livro que não existe ou tenta passar um cartão com saldo insuficiente para ver se o sistema barra a operação corretamente.
- **Manutenção e Evolução:** Seis meses depois, o dono da livraria decide vender e-books. Você precisará atualizar o sistema para suportar arquivos digitais e não apenas livros físicos.

4. Principais Participantes no Exemplo

- **Administrador de Banco de Dados (DBA):** É quem vai garantir que, se 100 clientes tentarem comprar o último exemplar de um livro digital ao mesmo tempo, o banco de dados não trave e processe as informações com segurança, sem corromper o estoque.
- **Especialista de Domínio:** É o **gerente da livraria** ou o **chefe do estoque**. É a pessoa que melhor entende como funciona a rotina diária (ex: como os livros são catalogados por categoria ou como funciona a devolução de um livro com defeito). Ele ajuda o analista a entender as regras do negócio.
- **Cliente-Contratante:** É o **dono da livraria** (investidor). Ele quer que o sistema funcione para aumentar os lucros e reduzir perdas, mas talvez ele não saiba os detalhes operacionais de como o estoquista recebe as caixas de livros na área de carga.
- **Avaliadores de Qualidade:** São os profissionais que vão testar o sistema antes de entregar ao dono da livraria, garantindo que o botão de "Finalizar Compra" realmente funcione e que o sistema seja rápido e seguro.

Com base na estrutura de estudos comuns de **Análise e Desenvolvimento de Sistemas (Engenharia de Software)** e seguindo estritamente a didática do resumo da Unidade 1, apresento o resumo conceitual da **Unidade 2 - Ciclos de Vida e Modelos de Processo de Software**, seguido de sua aplicação prática.

Unidade 2 - Ciclos de Vida e Modelos de Processo

1. O que é um Processo de Software?

Um processo de software é um conjunto estruturado de atividades, rotinas e responsabilidades necessárias para construir um sistema do zero ou evoluir um sistema existente. O objetivo de adotar um processo padronizado é trazer estabilidade, controle e previsibilidade a um ambiente de desenvolvimento.

2. Abordagens de Ciclos de Vida

Os modelos de processo determinam a ordem em que as atividades de engenharia (requisitos, análise, projeto, código e testes) serão executadas:

- **Modelo Cascata (Linear ou Tradicional):** As fases do projeto ocorrem de forma estritamente sequencial, como uma cascata. Uma etapa só se inicia quando a anterior está 100% concluída.
 - *Vantagem:* Fácil gerenciamento e cronogramas muito claros.
 - *Desvantagem:* Alta rigidez. Descobrir um erro de requisitos na fase de testes pode exigir que todo o projeto seja reiniciado, tornando as mudanças extremamente caras.
- **Modelo Iterativo e Incremental:** O sistema é quebrado em partes menores (incrementos). A cada ciclo (iteração), uma parte funcional do software é analisada, projetada, codificada e testada, sendo entregue progressivamente ao cliente.
 - *Vantagem:* Permite correções de rumo rápidas e o cliente vê o sistema funcionando desde o início.
- **Modelos Ágeis (Ex: Scrum, XP):** Focam na colaboração contínua com o usuário, respostas rápidas a mudanças e redução de documentações burocráticas, priorizando o software funcionando.

3. Gerenciamento de Riscos e Prototipação

- **Prototipação:** É a criação de versões preliminares e simplificadas do software (telas funcionais ou desenhos simulados) para que o usuário possa interagir e validar a interface e as regras de negócio. Ela ajuda a capturar requisitos ocultos antes que o código real comece a ser escrito.
- **Análise de Riscos:** Identificação antecipada de fatores que podem fazer o projeto falhar (falta de pessoal técnico, atrasos de fornecedores, complexidade tecnológica além do esperado). Processos modernos (como o Modelo Espiral) colocam a avaliação de riscos no centro de cada decisão do projeto.

Exemplo Prático: O Aplicativo de Delivery da Hamburgueria

Para entender como essas diferentes abordagens mudam a realidade de um projeto, imagine que fomos contratados para criar um **Aplicativo de Delivery para uma hamburgueria local**.

1. Se utilizássemos o Modelo Cascata (Tradicional):

- Passaríamos 2 meses apenas escrevendo documentos com todos os requisitos possíveis (cupons, rotas de motoboy, formas de pagamento, relatórios mensais).
- Depois, mais 1 mês desenhando a arquitetura e 3 meses programando tudo em segredo.
- **O Risco:** No 6º mês, ao entregar o aplicativo pronto, o dono da hamburgueria diz: *"Ficou ótimo, mas agora o mercado exige pagamento via Pix por aproximação direto na maquininha, e nosso contrato não prevê isso"*. Para alterar isso agora no modelo Cascata, o custo e o impacto no código seriam gigantescos, pois a fase de análise já fechou há meses.

2. Se utilizássemos o Modelo Iterativo e Incremental:

Em vez de tentar entregar o aplicativo inteiro de uma vez, dividiríamos o projeto em ciclos (incrementos):

- **Iteração 1:** Focamos apenas em permitir que o cliente veja o cardápio e monte o carrinho. Entregamos essa parte em 3 semanas.
- **Iteração 2:** Adicionamos a funcionalidade de fazer o login e realizar o pagamento. O cliente valida.
- **Iteração 3:** Implementamos o painel de rotas do motoboy e a cozinha.
- **O Benefício:** Se o mercado mudar ou o cliente quiser alterar a forma de pagamento, ajustamos isso no início da Iteração 2, sem perder o trabalho do projeto inteiro.

3. Aplicando a Prototipação no aplicativo:

Antes de digitar qualquer linha de código no computador, o analista desenha as telas do aplicativo em uma ferramenta de design (ou até no papel) e mostra ao chapeiro e ao atendente da hamburgueria.

Ao simular o clique no botão "Enviar para a Cozinha", o chapeiro avisa: *"A letra do pedido ficou muito pequena para ler de longe enquanto estou na chapa"*. Mudamos o protótipo em minutos. Se descobríssemos isso apenas com o aplicativo pronto e codificado, o retrabalho seria custoso.

4. Participantes da Unidade 1 atuando no Ciclo de Vida da Unidade 2:

- **O Especialista de Domínio (o chapeiro/atendente)** participa ativamente validando o *protótipo* das telas.

- O **Cliente-Contratante (dono da hamburgueria)** avalia os resultados ao final de cada *iteração* para garantir que o dinheiro investido está gerando o produto esperado.

Seguindo o fluxo lógico de uma ementa clássica de Engenharia de Software e Análise de Sistemas, após compreendermos os conceitos gerais (Unidade 1) e os modelos de processo e ciclos de vida (Unidade 2), entramos em uma das fases mais críticas e práticas de qualquer projeto: a **Unidade 3 - Engenharia de Requisitos**.

Abaixo está o resumo conceitual detalhado, seguido de sua aplicação prática.

Unidade 3 - Engenharia de Requisitos

1. O que são Requisitos de Software?

Um requisito é uma descrição de uma condição, capacidade ou funcionalidade que o sistema deve possuir para resolver um problema real do cliente ou atingir um objetivo de negócio. É a base de contrato entre a equipe de desenvolvimento e quem está contratando o software.

2. Classificação dos Requisitos

Os requisitos são amplamente divididos em três categorias principais:

- **Requisitos Funcionais (RF):** Descrevem as ações que o sistema deve ser capaz de realizar. São as funcionalidades diretas com as quais o usuário interage (o "o que" o sistema faz).
- **Requisitos Não Funcionais (RNF):** Descrevem propriedades, restrições, qualidades ou características globais que o sistema deve apresentar (o "como" o sistema faz). Estão ligados a desempenho, segurança, confiabilidade, portabilidade e usabilidade.
- **Regras de Negócio (RN):** Não são funcionalidades do software em si, mas sim leis, políticas, faturamentos ou restrições do mundo real que a empresa possui e que o software é obrigado a obedecer.

3. O Processo da Engenharia de Requisitos

Para gerenciar os requisitos sem que eles se percam ou mudem sem controle, o analista segue quatro subetapas fundamentais:

1. **Elicitação (ou Levantamento):** Fase de descoberta. O analista usa técnicas (entrevistas, questionários, observação do trabalho do usuário ou *brainstorming*) para coletar o que o cliente precisa.
2. **Análise e Negociação:** Fase onde os requisitos coletados são checados em busca de conflitos ou impossibilidades técnicas. Se o cliente pede algo que estoura o orçamento ou o tempo, é feita uma negociação para definir prioridades.
3. **Especificação (Documentação):** Tradução das necessidades em um documento formal (como a Documentação de Requisitos ou Histórias de Usuário) para que os programadores saibam exatamente o que construir.
4. **Validação:** O momento em que o analista revisa o documento escrito junto com os *stakeholders* para garantir que o que foi documentado é exatamente o que eles queriam dizer.

Exemplo Prático: O Sistema de Agendamento de uma Clínica Médica

Para ilustrar a Engenharia de Requisitos, imagine que fomos contratados para desenvolver o **Sistema de Agendamento e Prontuário para uma Clínica Médica**.

1. Identificando e Separando os Requisitos na Prática:

- **Exemplos de Requisitos Funcionais (O que o app faz):**

- **RF01:** O sistema deve permitir que o paciente agende uma consulta escolhendo o médico e o horário.
- **RF02:** O sistema deve enviar uma mensagem de confirmação automática por WhatsApp 24 horas antes da consulta.
- **RF03:** O médico deve conseguir emitir e assinar digitalmente uma receita médica durante o atendimento.

- **Exemplos de Requisitos Não Funcionais (Como o app se comporta):**

- **RNF01 (Segurança):** Todos os dados dos prontuários médicos dos pacientes devem ser criptografados para garantir a privacidade dos dados.
- **RNF02 (Desempenho):** A busca por horários disponíveis de um médico não deve demorar mais do que 2 segundos para carregar na tela.
- **RNF03 (Portabilidade):** O sistema deve funcionar perfeitamente tanto em computadores (desktop) quanto em celulares (Android e iOS).

- **Exemplos de Regras de Negócio (A lei da clínica que o código deve respeitar):**

- **RN01:** Um paciente só pode desmarcar ou reagendar uma consulta sem custo se o fizer com até 4 horas de antecedência.
- **RN02:** Médicos plantonistas não podem ter mais de 12 horas seguidas de consultas agendadas no mesmo dia.

2. O Processo de Engenharia de Requisitos na Clínica:

- **Na Elicitação:** O analista de sistemas passa a tarde na recepção da clínica observando as secretárias atenderem telefonemas e anotando como elas organizam a agenda de papel atual.
- **Na Análise e Negociação:** O dono da clínica pede: *"Quero que o sistema use Inteligência Artificial para prever se o paciente vai faltar baseado no histórico de trânsito da cidade dele"*. O analista analisa o prazo, vê que isso atrasaria o projeto em 6 meses e *negocia*: *"Podemos deixar isso para uma versão futura e focar agora no agendamento básico para inaugurarmos a clínica no prazo?"*. O cliente aceita.
- **Na Especificação:** O analista escreve o documento detalhando o comportamento de cada tela de agendamento para os programadores e o DBA (Administrador de Banco de Dados) criarem a estrutura correta.
- **Na Validação:** O analista senta com a recepcionista-chefe (nossa *Especialista de Domínio*) e lê os requisitos com ela: *"Aqui diz que quando o paciente faltar, a vaga reabre imediatamente no site. É isso mesmo?"*. Ela valida e assina o documento dando o sinal verde para a equipe começar a programar (usando o modelo *Incremental* que vimos na Unidade 2!).

Seguindo o fluxo natural da Engenharia de Software, após levantarmos e especificarmos detalhadamente o que o sistema precisa fazer (Unidade 3), entramos na fase de dar forma e estrutura a essa solução. Chegamos à **Unidade 4 - Arquitetura e Modelagem Estrutural de Sistemas**.

Aqui, o foco sai do "o que" o cliente precisa e entra no "como" a equipe técnica vai organizar as peças do software para que ele seja seguro, expansível e fácil de manter.

Unidade 4 - Arquitetura e Modelagem Estrutural

1. O que é Arquitetura de Software?

A arquitetura de software é o plano de alto nível que define a estrutura global do sistema. Ela determina quais serão os grandes blocos ou componentes do software, como eles se comunicarão entre si e quais tecnologias e padrões serão adotados para garantir os Requisitos Não Funcionais (como segurança e desempenho).

2. Padrões Arquiteturais Comuns

Para não reinventar a roda, os arquitetos utilizam padrões consagrados pelo mercado:

- **Arquitetura em Camadas (Layered Architecture):** Divide o sistema em fatias de responsabilidade (geralmente: Interface com o Usuário, Lógica de Negócio e Banco de Dados). Uma camada só conversa com a camada imediatamente abaixo dela, o que organiza o código e isola problemas.
- **Cliente-Servidor:** O sistema é dividido entre quem pede o serviço (Cliente/Aplicativo) e quem processa a informação e guarda os dados (Servidor).
- **Microserviços:** Em vez de um único sistema gigante (Monólito), o software é quebrado em vários mini-sistemas independentes que rodam sozinhos e se comunicam pela rede. Se um quebrar, os outros continuam funcionando.

3. Introdução à Modelagem Estrutural

Enquanto a arquitetura define o "esqueleto" macro, a modelagem estrutural define o "esqueleto" micro de componentes específicos do sistema através de diagramas. É aqui que entra o uso prático das notações padronizadas que vimos na Unidade 1:

- **Diagrama de Componentes:** Mostra como os módulos de código, bibliotecas e arquivos de configuração estão organizados e interligados no projeto.
- **Diagrama de Implantação (Deployment):** Mapeia a parte física do sistema. Ele mostra em quais servidores, computadores ou dispositivos móveis cada pedaço do software será instalado e como as redes vão conectá-los.

Exemplo Prático: O Sistema de Matrículas de uma Escola de Idiomas

Para ilustrar a Arquitetura e a Modelagem Estrutural, imagine que fomos contratados para projetar a estrutura técnica de um **Sistema de Matrículas e Aulas Online para uma Escola de Idiomas**.

1. Aplicando o Padrão Arquitetural em Camadas:

Para garantir que o código não vire uma bagunça, o arquiteto divide o sistema da escola em 3 camadas:

1. **Camada de Apresentação (Interface):** É a tela bonita onde o aluno digita seus dados para se matricular. Ela não sabe *como* a matrícula é validada, apenas coleta os dados.
 2. **Camada de Negócio (Lógica):** É onde ficam as regras de negócio (ex: verificar se o aluno tem a idade mínima para a turma de inglês avançado). Ela recebe os dados da tela, processa e toma a decisão.
 3. **Camada de Dados (Persistência):** É a parte que conversa diretamente com o banco de dados controlado pelo DBA para salvar a matrícula aprovada.
- *Benefício prático:* Se a escola decidir mudar o visual do site (Camada 1), as regras de validação de turmas (Camada 2) e o banco de dados (Camada 3) permanecem intactos, sem precisar mexer em uma linha de código deles.

2. Visualizando com os Diagramas Estruturais:

- **No Diagrama de Componentes:** O analista desenha um bloco chamado `Modulo_Financeiro` (responsável por gerar os boletos da matrícula) e mostra que ele se conecta e depende de um componente externo chamado `API_Banco_X`. O programador olha para esse diagrama e sabe exatamente qual pedaço de código precisa criar e onde acoplá-lo.
- **No Diagrama de Implantação:** O analista desenha a infraestrutura física do projeto:
 - Um bloco representando o **Smartphone do Aluno** (onde roda o app de apresentação).
 - Uma linha conectando o celular, via Internet, ao **Servidor de Aplicação na Nuvem** (onde roda a nossa lógica de negócios).
 - Outra linha conectando esse servidor de aplicação ao **Servidor de Banco de Dados** dedicado (onde o DBA configurou o armazenamento seguro dos dados dos alunos).

Com essa estrutura bem desenhada e modelada, a equipe de desenvolvimento tem o mapa completo para começar a codificar as funcionalidades que foram levantadas lá na engenharia de requisitos.

Avançando na estrutura programática da sua apostila de **Análise e Modelagem de Sistemas**, após passarmos pelas bases conceituais, ciclos de vida e engenharia de requisitos, entramos no coração da modelagem prática e visual de um software.

A **Unidade 2** foca na **UML (Unified Modeling Language)** e o **ecossistema de Ferramentas CASE**, que servem para transcrever toda a estrutura teórica conversada com o cliente em diagramas padronizados para o mercado mundial.

Unidade 5 - Introdução à UML e Ferramentas CASE

1. O que é e para que serve a UML?

A *Unified Modeling Language* (Linguagem de Modelagem Unificada) não é uma linguagem de programação, mas sim uma **notação gráfica padronizada**. O seu principal objetivo é prover uma linguagem visual pronta para que equipes de desenvolvimento possam especificar, visualizar, construir e documentar os artefatos de um sistema.

- **Independência:** A UML é independente de processos de desenvolvimento e de linguagens de programação. O arquiteto pode desenhar o sistema na UML e a equipa pode optar por codificar em qualquer linguagem tecnológica.
- **Semântica Rica:** Ela possui uma vasta gama de símbolos gráficos, onde cada seta, tracejado ou caixa possui uma sintaxe e um significado bem definidos, evitando ambiguidades entre programadores.

2. As Visões dos Modelos na UML

O software é modelado como se fosse feito em camadas, pois um único diagrama não consegue demonstrar o sistema por completo. A UML divide as suas visões em duas grandes categorias principais:

- **Diagramas de Estrutura (Estáticos):** Mostram a composição fixa e a organização física/lógica dos componentes do sistema e os seus relacionamentos, independentemente do tempo (Ex: Diagramas de Componentes, Implantação e Classes).
- **Diagramas de Comportamento (Dinâmicos):** Mostram o comportamento dinâmico e o fluxo de dados dos objetos ao longo do tempo em resposta a estímulos (Ex: Diagramas de Atividades, Sequência e Casos de Uso).

3. Ferramentas CASE (Computer-Aided Software Engineering)

Construir diagramas complexos manualmente é inviável em projetos profissionais. As ferramentas CASE são softwares criados para **auxiliar engenheiros e analistas em todo o ciclo de vida do desenvolvimento**. Elas atuam de várias formas:

- **Documentação e Sincronismo:** Geram relatórios textuais que ficam automaticamente coerentes com os diagramas gráficos desenhados.
- **Análise e Validação:** Os editores testam se os diagramas respeitam as regras da notação UML, apontando erros de lógica antes da escrita do código.
- **Geração de Código e Engenharia Reversa:** Muitas ferramentas CASE conseguem ler os diagramas e gerar automaticamente o esqueleto das tabelas de banco de dados ou arquivos de código, bem como ler um código existente e desenhar o diagrama estrutural dele.

Exemplo Prático: A Modernização de uma Oficina Mecânica

Para ilustrar a aplicação da UML e das Ferramentas CASE, imagine que fomos contratados para automatizar e criar o sistema de uma **Oficina Mecânica que antes controlava tudo em papel**.

1. Aplicando as Visões da UML na Oficina:

Em vez de desenhar um mapa gigante e confuso da oficina, o analista utiliza a separação de visões da UML:

- **Visão de Comportamento (Dinâmica):** O analista cria um *Diagrama de Atividades* (fluxograma avançado) para mapear o processo vivo do carro dentro da oficina: O carro chega $\rightarrow$ O mecânico faz a vistoria $\rightarrow$ O sistema gera o orçamento $\rightarrow$ O cliente aprova pelo WhatsApp $\rightarrow$ Inicia o reparo.
- **Visão de Estrutura (Estática):** O analista desenha um *Diagrama de Implantação* para mostrar a infraestrutura física : os tablets que os mecânicos usam na oficina física ligando-se ao servidor central hospedado na nuvem, onde o banco de dados guarda o histórico de peças.

2. Utilizando uma Ferramenta CASE no Projeto:

Para gerir este projeto, o analista abre um software CASE (como o *Enterprise Architect*, *Visual Paradigm* ou *ArgoUML*).

- **Evitando Erros:** Se o analista tentar conectar por engano um botão da tela diretamente a uma tabela restrita do banco de dados sem passar pela lógica de segurança, a ferramenta CASE avisa imediatamente: *"Esta ligação quebra as regras semânticas da arquitetura padronizada"*. O erro é corrigido na hora.
- **Agilizando a Programação:** Assim que o analista termina de desenhar as regras de negócio de como uma "Ordem de Serviço" deve ser calculada (com descontos e taxas de mão de obra), ele clica em um botão na ferramenta CASE. A ferramenta lê aquele gráfico e gera automaticamente a estrutura do banco de dados estruturado para o DBA e o esqueleto inicial de arquivos de código para os programadores começarem a trabalhar.

Com isso, o projeto ganha uma velocidade de execução muito superior se comparado à época em que os diagramas eram rascunhados soltos em quadros ou ferramentas de desenho comum.

Como as ferramentas CASE e a UML variam em termos de aplicação e recursos de automação dentro de uma ferramenta, a **Unidade 2** costuma classificar os "cases" (as ferramentas de engenharia de software auxiliada por computador) em três grandes categorias de mercado, divididas de acordo com o momento do ciclo de vida do projeto em que elas dão suporte:

1. Ferramentas Upper-CASE (Foco no Planejamento e Requisitos)

Essas ferramentas dão suporte às fases iniciais do projeto, focando no levantamento de requisitos, análise de negócios e modelagem conceitual de alto nível. Elas ajudam o analista e o especialista de domínio a organizarem as ideias antes de qualquer desenho técnico complexo.

- **O que fazem:** Permitem desenhar diagramas de fluxo de dados (DFDs), mapas conceituais e organizar a documentação de requisitos.
- **Exemplo de aplicação prática:** Ferramentas voltadas para *Business Process Management* (BPMN), como o **Bizagi Modeler**, ou softwares focados puramente em mapeamento de escopo e diagramação inicial, onde o objetivo é validar o entendimento do negócio junto ao cliente-contratante.

2. Ferramentas Lower-CASE (Foco na Execução e Construção)

Essas ferramentas atuam nas fases finais do ciclo de vida do desenvolvimento, dando suporte direto à codificação, geração de scripts de banco de dados, testes automatizados e implantação no servidor.

- **O que fazem:** Elas pegam especificações técnicas e as traduzem em arquivos prontos para execução. É onde o DBA (Administrador de Banco de Dados) e os programadores mais encontram automação direta.
- **Exemplo de aplicação prática:** Ferramentas de modelagem de banco de dados, como o **MySQL Workbench** ou **ERwin Data Modeler**. O profissional desenha a estrutura física das tabelas e a ferramenta gera automaticamente todo o código SQL necessário para criar o banco de dados real.

3. Ferramentas Integrated-CASE ou I-CASE (Foco no Ciclo Completo)

São as soluções mais robustas e completas do mercado mundial, pois cobrem o ciclo de vida de ponta a ponta — integrando as capacidades de planejamento das ferramentas *Upper* com a automação de código das ferramentas *Lower*.

- **O que fazem:** Permitem que o analista desenhe toda a arquitetura de sistemas e diagramas UML. Elas validam a consistência dos modelos, apontam erros de lógica e oferecem recursos avançados de **Engenharia Direta** (transformar gráficos em código-fonte) e **Engenharia Reversa** (transformar códigos existentes de volta em gráficos).
- **Exemplos de mercado mais comuns:**
 - **Enterprise Architect (Sparx Systems):** Uma das ferramentas industriais mais utilizadas para modelagem UML completa, gestão de requisitos e geração de código multicamadas.
 - **Visual Paradigm:** Extremamente popular no meio acadêmico e profissional por sua forte capacidade de integração com ambientes de desenvolvimento (IDEs) e mapeamento ágil de histórias de usuário.
 - **IBM Rational Software Architect / Rose:** Histórico pacote de ferramentas corporativas focado em engenharia de software rigorosa sob os padrões da UML.
 - **ArgoUML / Astah (antigo JUDE):** Alternativas focadas na modelagem ágil e rápida de diagramas estáticos e dinâmicos.

Trabalho(valor 10,0 pontos)

1. Por que a atividade de modelagem é considerada uma ferramenta estratégica para evitar retrabalho, prejuízos financeiros e estresse nas fases avançadas do desenvolvimento de um software?
2. De acordo com os autores clássicos Booch, Rumbaugh e Jacobson, a escolha dos modelos influencia diretamente a abordagem de um problema. Explique o que significa esse princípio.
3. Pensando na necessidade de alternar entre uma visão panorâmica (macro) e o detalhamento técnico (micro) de um sistema, qual princípio de modelagem aborda essa variação de precisão?
4. Explique por que sistemas complexos e robustos não podem ser explicados ou representados por meio de um único modelo isolado.
5. Embora o uso de notações gráficas padronizadas exija um aprendizado inicial por parte dos leitores, qual é o principal benefício que elas trazem para a documentação técnica de um projeto?
6. Diferencie as atividades de **Levantamento de Requisitos** da atividade de **Projeto (Desenho)** dentro do ciclo de engenharia de software.
7. No contexto prático de desenvolvimento, qual é a diferença fundamental entre as atribuições de um **Especialista de Domínio** e as de um **Cliente-Contratante**?
8. Qual é a responsabilidade principal do Administrador de Banco de Dados (DBA) no processo de desenvolvimento de sistemas?
9. Em pequenas empresas ou microempresas, como costuma ocorrer a divisão dos papéis e responsabilidades apresentados na Unidade 1?

Perguntas de Fixação: Unidade 2 – Ciclos de Vida, UML e Ferramentas CASE

10. O que é um **Processo de Software** e qual o principal objetivo de adotar um processo padronizado?
11. Apresente a principal característica do **Modelo Cascata** e explique por que a alta rigidez desse modelo pode tornar as mudanças de requisitos muito caras.
12. Como funciona a entrega de um software se a equipe optar pelo **Modelo Iterativo e Incremental**? Qual a sua principal vantagem em relação ao modelo tradicional?
13. O que é a técnica de **Prototipação** e de que forma ela auxilia os analistas a capturarem requisitos ocultos antes do início da escrita do código real?
14. A UML (*Unified Modeling Language*) é classificada como uma linguagem de programação? Justifique sua resposta abordando o conceito de independência tecnológica da notação.
15. A UML organiza seus diagramas em duas grandes categorias: **Visões Estáticas (Estruturais)** e **Visões Dinâmicas (Comportamentais)**. Diferencie o objetivo principal de cada uma das visões.
16. O que são **Ferramentas CASE** e de que maneira elas auxiliam os engenheiros de software na validação de diagramas e na geração automática de códigos?
17. As ferramentas CASE tem três categorias: *Upper-CASE*, *Lower-CASE* e *Integrated-CASE*. Explique o foco de atuação de cada uma delas e dê um exemplo prático de aplicação para cada tipo.
18. No contexto das ferramentas *I-CASE*, diferencie os conceitos de **Engenharia Direta** e **Engenharia Reversa**.